Video 12 — Slices in Golang (Deep Explanation)

What is a Slice?

A **slice** is a **dynamic array** — like an array but without a fixed size. It can grow and shrink automatically as you add or remove elements.

Array vs Slice — Core Difference:

Feature	Array	Slice
Size	Fixed at creation	Dynamic — grows automatically
Size declaration	Required `[4]int`	Not required `[]int`
Usage in Go	Rare	**Extremely common**
Flexibility	Low	High

Slices are the most used data structure in Go — you will use them everywhere

---

File Setup

```
golang-course/
└── 9-slices/
    └── slices.go
```

```go
package main

import "fmt"

func main() {
    // code goes here
}
```

Way 1 — Declaring a Nil Slice

```
var nums []int
fmt.Println(nums)          // Output: []
fmt.Println(nums == nil)   // Output: true
fmt.Println(len(nums))     // Output: 0
```

Breaking it down:

Part	Meaning
`var nums`	declare variable
`[]int`	slice of integers — no size given
`nil`	uninitialized slice is nil by default

What is `nil`?

- `nil` in Go is equivalent to `null` in other languages
- An uninitialized slice has no memory allocated yet
- It's empty and `nil` until you put something in it

Way 2 — Using `make()` Function (Most Common)

```
var nums = make([]int, 2)
fmt.Println(nums)          // Output: [0 0]
fmt.Println(nums == nil)   // Output: false
fmt.Println(len(nums))     // Output: 2
fmt.Println(cap(nums))     // Output: 2
```

Breaking down `make()`:

```
make([]int, 2)
//    type  length
```

Parameter	Meaning
`[]int`	type of slice — integer
`2`	initial length — creates 2 slots with zero values

- When you use `make()`, the slice is **no longer nil** — it's initialized
- Zero values are automatically filled: `0` for int, `false` for bool, `""` for string

With initial capacity (3rd parameter):

```
var nums = make([]int, 0, 5)
fmt.Println(cap(nums))  // Output: 5
```

Parameter	Meaning
`[]int`	type
`0`	initial length
`5`	initial capacity

Understanding Length vs Capacity

These are two different concepts — very important:

```
nums := make([]int, 2, 5)
fmt.Println(len(nums))  // 2
fmt.Println(cap(nums))  // 5
```

Concept	Meaning
Length (`len`)	How many elements are currently in the slice
Capacity (`cap`)	How many elements the slice can hold before needing to resize

Auto-Resizing (Doubling Algorithm):

When you add elements beyond the current capacity, Go **automatically doubles** the capacity:

```
nums := make([]int, 0, 5)
```

```
// Add elements 1, 2, 3, 4 → still fits in capacity 5
// Add element 5, 6 → exceeds 5 → capacity doubles to 10
// Add more → exceeds 10 → capacity doubles to 20
```

This is how dynamic arrays work internally — Go handles it automatically for you.

Way 3 — Short Declaration with Values

```
nums := []int{1, 2, 3}
fmt.Println(nums)    // Output: [1 2 3]
fmt.Println(len(nums))  // 3
fmt.Println(cap(nums))  // 3
```

- No `make()` needed
- Just declare and initialize with values directly
- Length and capacity both equal number of elements given

Adding Elements — `append()`

The most commonly used slice operation:

```
nums := []int{}           // empty slice

nums = append(nums, 1)     // add 1
nums = append(nums, 2)     // add 2
nums = append(nums, 3, 4)  // add multiple at once

fmt.Println(nums)         // Output: [1 2 3 4]
```

Important syntax rule:

```
nums = append(nums, value)
//  ↑ must reassign back to same variable
```

- `append()` returns a **new slice** with the element added
- You must **assign it back** to the variable
- Not doing this is a common beginner mistake

append() Syntax:

```
append(slice, element)          // add one element
append(slice, elem1, elem2)     // add multiple elements
```

Setting Elements by Index

```
nums := make([]int, 2)  // creates [0, 0]
nums[0] = 3
nums[1] = 5
fmt.Println(nums)       // Output: [3 5]
```

⚠️ Index Out of Range Error:

```
nums := make([]int, 0)  // length is 0
nums[0] = 3             // ❌ ERROR: index out of range
```

- You can only set an index that already exists within the current **length**
- If length is 0, no index exists yet — use `append()` instead

Copying a Slice — `copy()`

```
nums1 := []int{2}
nums2 := make([]int, len(nums1))  // destination must have space

copy(nums2, nums1)  // copy(destination, source)

fmt.Println(nums1)  // [2]
fmt.Println(nums2)  // [2]
```

Important rules for `copy()`:

- First argument = **destination**
- Second argument = **source**
- Destination slice must have **enough length** to hold the copied elements
- If destination length is 0 → nothing gets copied

```
// ❌ Won't work — destination has length 0
```

```
nums2 := make([]int, 0)
copy(nums2, nums1)  // nothing copied!

// ✅ Works — destination has enough length
nums2 := make([]int, len(nums1))
copy(nums2, nums1)  // copied!
```

Slice Operator `[low:high]`

This lets you extract a **portion** of a slice:

```
nums := []int{1, 2, 3}

fmt.Println(nums[0:2])  // [1 2] — index 0 and 1 (2 excluded)
fmt.Println(nums[0:1])  // [1]   — index 0 only (1 excluded)
fmt.Println(nums[1:3])  // [2 3] — index 1 and 2 (3 excluded)
```

Rule: `[low:high]`

- `low` = starting index (included)
- `high` = ending index (**excluded**)
- Returns elements from `low` up to but **not including** `high`

Omitting values:

```
nums := []int{1, 2, 3}

fmt.Println(nums[:2])   // [1 2]   — from beginning to index 2 (excluded)
fmt.Println(nums[1:])   // [2 3]   — from index 1 to end
fmt.Println(nums[:])    // [1 2 3] — entire slice
```

Syntax	Meaning
`nums[0:2]`	index 0 to 1 (2 excluded)
`nums[:2]`	start to index 1 (2 excluded)
`nums[1:]`	index 1 to end

`nums[:]`	entire slice

Slices Package — `slices.Equal()`

Go has a built-in `slices` package for working with slices:

```
import "slices"

nums1 := []int{1, 2}
nums2 := []int{1, 2}
nums3 := []int{1, 4}

fmt.Println(slices.Equal(nums1, nums2))  // true
fmt.Println(slices.Equal(nums1, nums3))  // false
```

How `slices.Equal()` works:

- Compares element by element in order
- Returns `true` only if **all elements match** in the same order
- Returns `false` as soon as one mismatch is found
- Returns a **boolean** value

Other methods in the `slices` package:

- `slices.Clone()` — clone a slice
- `slices.Contains()` — check if element exists
- `slices.Index()` — find index of element
- And many more

2D Slices

Just like 2D arrays — a slice of slices:

```
nums := [][]int{
    {1, 2, 3},
    {4, 5, 6},
}
```

```
fmt.Println(nums)  // [[1 2 3] [4 5 6]]
```

- No fixed size needed for either dimension
- Both dimensions are dynamic
- Useful for grids, matrices, tables

Full Code Example

```
package main

import (
    "fmt"
    "slices"
)

func main() {

    // Way 1 — Nil slice
    var nums []int
    fmt.Println(nums == nil)    // true
    fmt.Println(len(nums))      // 0

    // Way 2 — make()
    nums2 := make([]int, 0, 5)
    fmt.Println(cap(nums2))     // 5

    // Way 3 — short declaration
    nums3 := []int{1, 2, 3}
    fmt.Println(nums3)          // [1 2 3]

    // append()
    nums4 := []int{}
    nums4 = append(nums4, 1)
    nums4 = append(nums4, 2, 3)
    fmt.Println(nums4)          // [1 2 3]

    // copy()
    src := []int{10, 20}
    dst := make([]int, len(src))
    copy(dst, src)
    fmt.Println(dst)            // [10 20]
```

```go
    // slice operator
    s := []int{1, 2, 3, 4, 5}
    fmt.Println(s[1:3])        // [2 3]
    fmt.Println(s[:2])         // [1 2]
    fmt.Println(s[3:])         // [4 5]

    // slices package
    a := []int{1, 2, 3}
    b := []int{1, 2, 3}
    fmt.Println(slices.Equal(a, b))  // true

    // 2D slice
    grid := [][]int{{1, 2}, {3, 4}}
    fmt.Println(grid)          // [[1 2] [3 4]]
}
```

Running the Program

go run 9-slices/slices.go

Zero Values in Slices (Reminder)

When using `make()` with a length, slots are filled with zero values:

Type	Zero Value
`int`	`0`
`bool`	`false`
`string`	`""`
`float64`	`0`

To avoid pre-filled zero values, use `make([]int, 0)` — length 0, then append.

Summary Table

Operation	Code
Declare nil slice	`var nums []int`
Make with length	`make([]int, 2)`
Make with length + capacity	`make([]int, 0, 5)`
Short declaration	`nums := []int{1, 2, 3}`
Append element	`nums = append(nums, val)`
Set by index	`nums[0] = 10`
Get length	`len(nums)`
Get capacity	`cap(nums)`
Copy slice	`copy(dst, src)`
Slice operator	`nums[1:3]`
Compare slices	`slices.Equal(a, b)`
2D slice	`[][]int{{1,2},{3,4}}`

Key Takeaways

1. Slices are **dynamic arrays** — no fixed size needed
2. Slices are the **most used data structure** in Go — master them well
3. Uninitialized slice is **nil** — use `make()` to initialize
4. **Length** = current elements, **Capacity** = max before resize
5. When capacity is exceeded, Go **doubles the capacity** automatically
6. Always **reassign** when using `append()` → `nums = append(nums, val)`
7. `copy()` needs destination to have **enough length** — otherwise nothing copies
8. Slice operator `[low:high]` — low is **included**, high is **excluded**
9. The `slices` package provides useful methods like `Equal()`, `Clone()`, etc.
10. 2D slices work just like 2D arrays but are **fully dynamic**

Yes, that is my explanation! Looking at it carefully, there are **a few things I added** that weren't in the transcript, and **everything from the transcript is covered**. But let me explain the concepts I explained in **even more depth** with things I may have glossed over:

---

1. Why `make([]int, 0)` instead of `make([]int, 2)`?

This is a subtle but important point the instructor mentioned:

```
// ❌ This creates unwanted zero values
nums := make([]int, 2)
nums = append(nums, 1)
nums = append(nums, 2)
fmt.Println(nums)  // [0 0 1 2] — not what you wanted!

// ✅ This starts clean
nums := make([]int, 0)
nums = append(nums, 1)
nums = append(nums, 2)
fmt.Println(nums)  // [1 2] — correct!
```

Why does this happen?

- When you say `make([]int, 2)`, Go creates 2 slots immediately filled with `0`
- When you then `append()`, it adds **after** those existing zero slots
- So your actual values end up at index 2, 3 — not 0, 1
- This is why the instructor said **use length 0** when you plan to append later

---

2. The Capacity Doubling — Shown More Clearly

The instructor demonstrated this live. Let me show it step by step:

```
nums := make([]int, 0, 5)
// Length: 0, Capacity: 5

nums = append(nums, 1)  // Length: 1, Capacity: 5
nums = append(nums, 2)  // Length: 2, Capacity: 5
nums = append(nums, 3)  // Length: 3, Capacity: 5
```

```
nums = append(nums, 4)  // Length: 4, Capacity: 5
nums = append(nums, 5)  // Length: 5, Capacity: 5
nums = append(nums, 6)  // Length: 6, Capacity: 10 ← DOUBLED!

// Continuing...
nums = append(nums, 7, 8, 9, 10)  // Length: 10, Capacity: 10
nums = append(nums, 11)           // Length: 11, Capacity: 20 ← DOUBLED AGAIN!
```

Why does Go double it?

- Doubling is more efficient than adding one slot at a time
- If it added one slot every time, it would constantly allocate new memory — very slow
- Doubling means fewer resizes needed as the slice grows
- This is a standard **computer science algorithm** called **amortized resizing**

---

3. What Actually Happens During Auto-Resize

When Go doubles the capacity behind the scenes:

```
Old slice: [1, 2, 3, 4, 5]  capacity=5
↓
Go creates a NEW bigger array in memory: capacity=10
↓
Copies all old elements into new array
↓
Adds new element at the end
↓
Returns new slice pointing to new memory location
```

- Go is **allocating new memory** and **copying everything over**
- This happens automatically — you don't see it
- This is why `append()` must always be **reassigned** — it may return a completely new slice in memory

---

4. Why `append()` Must Be Reassigned — Deeper Explanation

This confused a lot of beginners. Here's why:

```
nums := []int{1, 2, 3}
append(nums, 4)       // ❌ WRONG — result is thrown away
fmt.Println(nums)     // [1 2 3] — 4 never got added!

nums = append(nums, 4) // ✅ CORRECT
fmt.Println(nums)     // [1 2 3 4]
```

Why does this happen?

- `append()` does NOT modify the original slice
- It creates and **returns a brand new slice**
- If you don't capture that return value, it's just lost
- Think of it like: `result = calculate(5)` — if you don't store the result, it's gone

---

5. The `copy()` Function — Why Destination Needs Length

The instructor showed this bug live:

```
src := []int{1, 2, 3}

// ❌ Bug — nothing gets copied
dst := make([]int, 0)
copy(dst, src)
fmt.Println(dst)  // [] — empty! copy failed silently

// ✅ Correct
dst := make([]int, len(src))  // length = 3
copy(dst, src)
fmt.Println(dst)  // [1 2 3]
```

Why does destination need length?

- `copy()` only copies **up to the length of the destination**
- If destination length is 0 → it copies 0 elements → nothing happens
- It doesn't error out — it just silently copies nothing
- This is a tricky bug that's easy to miss

The rule:

Number of elements copied = min(len(dst), len(src))

---

6. Slice Operator — Deeper Understanding

```
nums := []int{1, 2, 3, 4, 5}
//           0  1  2  3  4   ← indices

nums[1:3]
// starts at index 1 → value: 2
// ends before index 3 → value: 4 (excluded)
// Result: [2 3]
```

Visual representation:

```
Index:  0   1   2   3   4
Value:  1   2   3   4   5
          ↑       ↑
        low=1   high=3

Result: [2, 3]  (index 1 and 2 only, 3 is excluded)
```

All combinations:

```
nums[:]    // [1 2 3 4 5] — everything
nums[2:]   // [3 4 5]     — from index 2 to end
nums[:3]   // [1 2 3]     — from start to index 2 (3 excluded)
nums[1:4]  // [2 3 4]     — index 1, 2, 3 (4 excluded)
```

---

7. Nil Slice vs Empty Slice — Important Difference

The instructor touched on this but it's worth being very clear:

```
// Nil slice
var nums []int
fmt.Println(nums == nil)   // true
fmt.Println(len(nums))     // 0
```

```go
// Empty slice (NOT nil)
nums2 := []int{}
fmt.Println(nums2 == nil)  // false
fmt.Println(len(nums2))    // 0

// Also empty but NOT nil
nums3 := make([]int, 0)
fmt.Println(nums3 == nil)  // false
fmt.Println(len(nums3))    // 0
```

Type	`== nil`	`len()`	Has Memory
`var nums []int`	`true`	`0`	No
`[]int{}`	`false`	`0`	Yes
`make([]int, 0)`	`false`	`0`	Yes

- Both nil and empty slices have length 0
- But only `var` declaration gives you a nil slice
- In practice, both behave similarly for most operations
- `append()` works on nil slices too

---

8. 2D Slice — More Depth

```go
nums := [][]int{
    {1, 2, 3},
    {4, 5, 6},
}
```

Accessing elements:

```go
fmt.Println(nums[0])     // [1 2 3] — first row
fmt.Println(nums[1])     // [4 5 6] — second row
fmt.Println(nums[0][1])  // 2 — row 0, column 1
fmt.Println(nums[1][2])  // 6 — row 1, column 2
```

Unlike 2D arrays, rows can have different lengths:

```
nums := [][]int{
    {1, 2},          // row 0 has 2 elements
    {3, 4, 5, 6},    // row 1 has 4 elements
}
// This is valid! Slices are flexible
```

Summary of What's Deeper Now

Concept	What I Added
`make([]int, 0)`	Why to use 0 length to avoid unwanted zeros
Capacity doubling	Step by step demonstration
Behind the scenes resize	Memory copy explanation
`append()` reassign	Why it must be reassigned — new slice returned
`copy()` silent fail	Why destination needs length — copies `min(dst,src)`
Slice operator	Visual index diagram
Nil vs Empty slice	3-way comparison table
2D slice	Element access + unequal row lengths

Everything else in your notes was already explained thoroughly! Let me know if anything is still unclear. 😊